

Recognition Model Architectural Variants: Comprehensive Comparison Report

Date: December 29, 2024 **Author:** Can Konuk **Experiment Duration:** ~3 hours total

Executive Summary

This report documents a systematic comparison of 14 architectural variants for the Contrastive Recognition Model, testing different combinations of: - Card encoders (standard vs enhanced) - Hand encoders (mean vs attention vs deepsets vs multiscale) - Task encoders (standard vs multihead) - Prediction heads (sigmoid vs embedding) - Loss functions (BCE vs focal)

Key Finding: The **enhanced_card** variant with sigmoid head achieves the best performance (R@5=0.247, MRR=0.335), providing a modest but consistent improvement over the baseline (R@5=0.243, MRR=0.325). Embedding-based prediction heads significantly underperform sigmoid heads.

1. Main Results Table

Sigmoid Head Variants (Successful)

Variant	R@5	R@10	MRR	Prob Ratio	Conv. Epoch
enhanced_card	0.247±0.01	0.312±0.01	0.335±0.02	4.6	83
baseline	0.243±0.01	0.308±0.01	0.325±0.02	15.9	77
attention_hand	0.243±0.01	0.311±0.01	0.328±0.01	13.3	62
multihead_task	0.241±0.02	0.310±0.01	0.318±0.02	17.1	84
enhanced+attention	0.237±0.01	0.304±0.01	0.316±0.02	15.4	92
deepsets_hand	0.236±0.02	0.304±0.03	0.310±0.03	12.8	95
multiscale_hand	0.233±0.02	0.307±0.01	0.303±0.02	10.7	85
focal_loss	0.231±0.02	0.312±0.01	0.318±0.03	10.0	55

Embedding Head Variants (Re-run after bug fix)

Variant	R@5	R@10	MRR
embedding_head	0.127	0.228	0.107
enhanced+attention+embed+focal	0.096	0.150	0.078
enhanced+deepsets+embed	0.089	0.154	0.075
full_enhanced	0.088	0.153	0.080
attention+multi+embed+focal	0.059	0.136	0.063
enhanced+attention+embed	0.058	0.121	0.065

2. Component-Wise Analysis

2.1 Card Encoder Comparison

Type	Mean R@5	Mean MRR	Notes
Enhanced	0.242	0.326	+1.7% R@5 improvement
Standard	0.238	0.317	Baseline performance

The enhanced card encoder adds: - Color embedding (4D: red/black distinction) - Numeric rank value (normalized 1-13)

Conclusion: Enhanced card encoding provides small but consistent improvement.

2.2 Hand Encoder Comparison

Type	Mean R@5	Mean MRR	Notes
Mean pooling	0.241	0.324	Simple and effective
Attention	0.240	0.322	Slight overhead, no gain
DeepSets	0.236	0.310	Underperforms mean
Multiscale	0.233	0.303	Worst performance

Conclusion: Simple mean pooling is optimal. More complex encoders don't help and may hurt generalization.

2.3 Task Encoder Comparison

Type	Mean R@5	Mean MRR	Notes
Standard	0.239	0.319	= mean(pos) - mean(neg)
Multihead	0.241	0.318	Similar performance, higher variance

Conclusion: Standard contrastive encoding is sufficient.

2.4 Prediction Head Comparison

Type	Mean R@5	Mean MRR	Notes
Sigmoid	0.239	0.319	Strong, consistent
Embedding	0.086	0.078	-64% degradation

Conclusion: Embedding-based prediction severely underperforms. The learned primitive embeddings + dot-product scoring approach is not effective for this task.

2.5 Loss Function Comparison

Type	Mean R@5	Mean MRR	Conv. Epoch
BCE	0.240	0.319	~80
Focal	0.231	0.318	55

Conclusion: Focal loss converges faster but with slightly worse final performance. BCE is preferred.

3. Key Insights

3.1 What Works

1. **Enhanced card encoding** provides consistent improvement by capturing color and numeric rank
2. **Simple mean pooling** is optimal for aggregating card embeddings
3. **Standard contrastive encoding** (= pos - neg) is effective
4. **Sigmoid prediction head** significantly outperforms embedding-based alternatives
5. **BCE loss** achieves best final performance

3.2 What Doesn't Work

1. **Attention-based hand encoding** - Adds complexity without benefit
2. **DeepSets/Multiscale pooling** - Underperforms simple mean pooling
3. **Embedding prediction heads** - 64% worse than sigmoid heads
4. **Focal loss** - Faster convergence but worse final accuracy
5. **Complex combinations** - Adding multiple enhancements doesn't compound benefits

3.3 Why Embedding Heads Fail

The embedding-based prediction head learns primitive embeddings and scores primitives via dot product with task embeddings. This approach fails because:

1. **Task embedding space mismatch** - The contrastive task embedding space may not align well with a dot-product scoring objective
2. **Sparse supervision** - Each task only uses 3-5 primitives out of 59, making it hard to learn meaningful primitive embeddings
3. **Optimization difficulty** - Joint optimization of task encoder and primitive embeddings is harder than a simple MLP

4. Recommended Configuration

Based on comprehensive testing, the **recommended configuration** is:

```
model = RecognitionModelVariant(  
    grammar=grammar,  
    card_encoder_type='enhanced',    # Best: adds color + rank value  
    hand_encoder_type='mean',        # Best: simple and effective  
    task_encoder_type='standard',     # Best: = mean(pos) - mean(neg)  
    prediction_head_type='sigmoid',   # Best: significantly better than embedding  
    loss_type='bce'                   # Best: better than focal loss  
)
```

Expected Performance: - R@5: 0.247 ± 0.01 - R@10: 0.312 ± 0.01 - MRR: 0.335 ± 0.02

5. Experimental Setup

5.1 Data

- **Training:** 44 pre-training rules (5-fold cross-validation)
- **Testing:** 45 catalogue rules
- **Hand size:** 5-6 cards per example
- **Examples per task:** 50 (25 positive, 25 negative)

5.2 Training

- **Optimizer:** Adam (lr=0.001)
- **Epochs:** 100 (early stopping with patience=20)
- **Batch size:** 8
- **Architecture:** 59 primitives, 64-dim embeddings

5.3 Metrics

- **R@k:** Fraction of ground truth primitives in top-k predictions
 - **MRR:** Mean Reciprocal Rank of ground truth primitives
 - **Prob Ratio:** Probability of correct primitives / probability of incorrect
-

6. Bugs Fixed During Experiments

Bug 1: Enum Value Handling

Location: `hand_to_tensors()` function **Issue:** Card suits/ranks are Enum objects, not strings. Direct lookup failed. **Fix:** Extract `.value` from Enum before lookup.

Bug 2: Embedding Collapse

Location: `card_mlp` in `RecognitionModelVariant` **Issue:** Final ReLU activation caused all embeddings to collapse to identical values. **Fix:** Removed final ReLU, allowing diverse embeddings.

Bug 3: Dimension Mismatch

Location: `encode_task_batched()` method **Issue:** When no pos/neg hands exist, returned 32-dim zeros instead of 64-dim (`task_dim`). **Fix:** Store `self.task_dim` and use it for zero tensor creation.

7. Files Modified/Created

File	Purpose
<code>dreamcoder_core/recognition_variants.py</code>	All architectural variants
<code>dreamcoder_core/contrastive_recognition_variants.py</code>	Previous baseline
<code>experiments/comprehensive_variant_comparison.py</code>	Main experiment runner
<code>experiments/run_embedding_variants.py</code>	Embedding head re-run
<code>experiments/diagnose_*.py</code>	Debugging scripts

8. Conclusions

1. **Simplicity wins** - Complex architectural additions rarely improve over well-tuned simple baselines
2. **Enhanced card encoding** is the one modification that provides consistent benefit
3. **Embedding prediction heads** are not suitable for this task - sigmoid MLPs are strongly preferred
4. **Standard contrastive encoding** (`= pos - neg`) is sufficient and effective
5. **Mean pooling** outperforms attention and DeepSets for hand encoding

The marginal gains from architectural changes ($0.243 \rightarrow 0.247$ R@5) suggest that further improvement may require: - Larger training datasets - Better primitive representations - Multi-task or transfer learning - Changes to the contrastive formulation itself